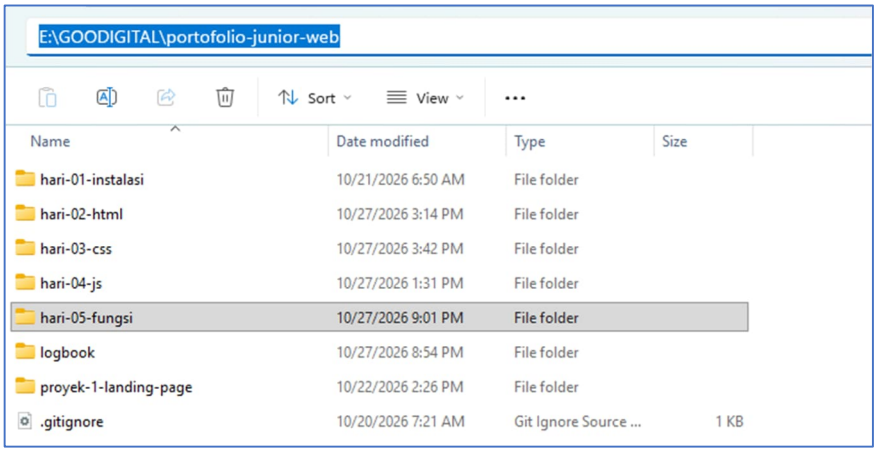
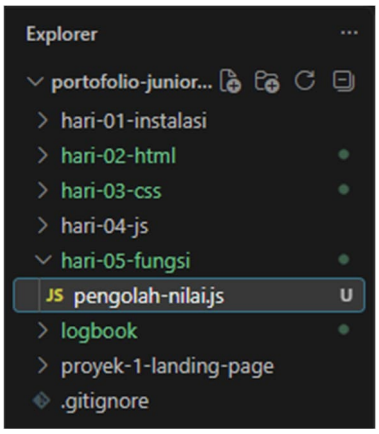
Modul Hari 5 - Fungsi, Array, DOM, dan Modularisasi (Proyek 2)

Lab 1 - Refactor Pengolah Nilai ke Fungsi

1. Menyiapkan Berkas Refactor

- Membuat folder hari-05-fungsi.
- Menyalin file pengolah-nilai.js.
- Menyiapkan berkas refactor fungsi.





2. Fungsi Pengolahan Nilai

- Membuat fungsi perhitungan statistik nilai.
- Membuat fungsi grade, status, dan validasi nilai.
- Menguji fungsi yang dibuat.

```
function hitungRataRata(daftar) {
  if (daftar.length === 0) return 0;

  return daftar.reduce(
    (jumlah, n) => jumlah + n,
    0
  ) / daftar.length;
}

function cariTertinggi(daftar) {
  return daftar.length === 0
    ? null
    : Math.max(...daftar);
}

function cariTerendah(daftar) {
  return daftar.length === 0
    ? null
    : Math.min(...daftar);
}

function tentukanGrade(nilai) {
  if (nilai >= 85) return 'A';
  if (nilai >= 70) return 'B';
  if (nilai >= 55) return 'C';
  return 'D';
}
```

3. Pengujian dengan console.assert

- Menambahkan pengujian sederhana pada fungsi.
- Memverifikasi hasil fungsi menggunakan console.assert.

```
console.assert(
  hitungRataRata([80, 90]) === 85,
  'rata-rata 80,90 harus 85'
);

console.assert(
  hitungRataRata([]) === 0,
  'rata-rata array kosong harus 0'
);

console.assert(
  tentukanGrade(85) === 'A',
  'nilai 85 harus A'
);

console.assert(
  tentukanGrade(69) === 'C',
  'nilai 69 harus C'
);

console.assert(
  nilaiValid(101) === false,
  '101 tidak valid'
);
```

```
Problems  Output  Debug Console  Terminal  Ports        node - hari-05-fungsi

PS E:\GOODIGITAL\portofolio-junior-web\hari-05-fungsi> node pengolah-nilai.js

  ===== PENGOLAH NILAI =====
  1. Input nilai siswa
  2. Tampilkan statistik
  3. Tampilkan predikat per siswa
  4. Ranking siswa
  5. Histogram grade
  6. Keluar
  Pilih menu (1-6):
```

4. Fungsi Prosedur

- Membuat fungsi tampilkanMenu().
- Membuat fungsi tampilkanLaporan().
- Memanggil fungsi prosedur dari main().

```
function tampilkanMenu() {
  console.log('\n===== PENGOLAH NILAI =====');
  console.log('1. Input nilai siswa');
  console.log('2. Tampilkan statistik');
  console.log('3. Tampilkan predikat per siswa');
  console.log('4. Ranking siswa');
  console.log('5. Histogram grade');
  console.log('6. Keluar');
}

function tampilkanLaporan(nama, nilai) {
  console.log(
    `${nama} | Nilai: ${nilai} | Grade: ${tentukanGrade(nilai)} | ${tentukanStatus(nilai)}`
  );
}
```

```
async function main() {
  const rl = readline.createInterface({ input, output });
  const daftarNama = [];
  const daftarNilai = [];
  let jalan = true;

  while (jalan) {
    tampilkanMenu();
    const pilihan = await rl.question('Pilih menu (1-6): ');
```

```
        const status =
          nilai >= BATAS_LULUS
            ? 'Lulus'
            : 'Tidak Lulus';

        tampilkanLaporan(nama, nilai);
```

5. JSDoc

- Menambahkan dokumentasi fungsi menggunakan JSDoc.
- Menjelaskan parameter dan nilai balik fungsi.
- Menguji program setelah penambahan dokumentasi.

```
/**
 * Menghitung rata-rata nilai.
 * @param {number[]} daftar
 * @returns {number}
 */

function hitungRataRata(daftar) {
  if (daftar.length === 0) return 0;

  return daftar.reduce(
    (jumlah, n) => jumlah + n,
    0
  ) / daftar.length;
}

/**
 * Mencari nilai tertinggi.
 * @param {number[]} daftar
 * @returns {number|null}
 */
```

Lab 2 - Array dan Method Array

1. Menyiapkan Data Peserta

- Membuat array of object.
- Menambahkan data peserta.

```
hari-05-fungsi > JS array-peserta.js > ...
1   const peserta = [
2     { id: 1, nama: 'Amri', kelas: 'Web', nilai: 97 },
3     { id: 2, nama: 'Badru', kelas: 'Web', nilai: 100 },
4     { id: 3, nama: 'Abdul', kelas: 'Data', nilai: 95 },
5     { id: 4, nama: 'Tamam', kelas: 'Data', nilai: 87 },
6     { id: 5, nama: 'Azzam', kelas: 'Web', nilai: 75 },
7     { id: 6, nama: 'Yaqin', kelas: 'Data', nilai: 56 },
8     { id: 7, nama: 'Quo', kelas: 'Data', nilai: 64 },
9     { id: 8, nama: 'Haura', kelas: 'Web', nilai: 99 },
10    { id: 9, nama: 'Aina', kelas: 'Data', nilai: 86 },
11    { id: 10, nama: 'Levan', kelas: 'Web', nilai: 84 },
12    { id: 11, nama: 'Dia', kelas: 'Web', nilai: 40 },
13  ];
```

2. Daftar Nama Peserta yang Lulus

- Menyaring peserta yang lulus menggunakan filter().
- Mengambil nama peserta menggunakan map().
- Menampilkan hasil seleksi peserta lulus.

```
const pesertaLulus = peserta
  .filter((p) => p.nilai >= 70)
  .map((p) => p.nama);

console.log('Peserta Lulus:');
console.log(pesertaLulus);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-05-fungsi> node array-peserta.js
Peserta Lulus:
[
  'Amri',  'Badru',
  'Abdul', 'Tamam',
  'Azzam', 'Haura',
  'Aina',  'Levan'
]
```

3. Rata-rata Nilai per Kelas

- Mengelompokkan peserta berdasarkan kelas.
- Menghitung rata-rata nilai tiap kelas.
- Menampilkan hasil perhitungan.

```
const kelasWeb = peserta.filter(
  (p) => p.kelas === 'Web'
);

const kelasData = peserta.filter(
  (p) => p.kelas === 'Data'
);

const rataWeb =
  kelasWeb.reduce(
    (jumlah, p) => jumlah + p.nilai,
    0
  ) / kelasWeb.length;

const rataData =
  kelasData.reduce(
    (jumlah, p) => jumlah + p.nilai,
    0
  ) / kelasData.length;

console.log('\nRata-rata per Kelas');
console.log('Web :', rataWeb.toFixed(2));
console.log('Data:', rataData.toFixed(2));
```

```
Problems  Output  Debug Console  Terminal  Ports   powershell - hari-05-fungsi

PS E:\GOODIGITAL\portofolio-junior-web\hari-05-fungsi> node array-peserta.js
Rata-rata per Kelas
Web : 82.50
Data: 77.60
```

4. Peserta dengan Nilai Tertinggi

- Menentukan peserta dengan nilai tertinggi menggunakan reduce().
- Menampilkan data peserta dengan nilai tertinggi.

```
const pesertaTerbaik = peserta.reduce(
  (tertinggi, p) =>
    p.nilai > tertinggi.nilai
      ? p
      : tertinggi
);

console.log('\nPeserta Nilai Tertinggi');
console.log(pesertaTerbaik);
```

```
Problems  Output  Debug Console  Terminal  Ports   powershell - hari-05-fungsi

PS E:\GOODIGITAL\portofolio-junior-web\hari-05-fungsi> node array-peserta.js
Peserta Nilai Tertinggi
{ id: 2, nama: 'Badru', kelas: 'Web', nilai: 100 }
```

5. Pencarian Peserta Berdasarkan Sebagian Nama

- Melakukan pencarian peserta menggunakan filter().
- Menggunakan includes() tanpa membedakan huruf besar dan kecil.
- Menampilkan hasil pencarian peserta.

```
const kataCari = 'am';

const hasilCari = peserta.filter(
  (p) =>
    p.nama.toLowerCase()
      .includes(kataCari.toLowerCase())
);

console.log('\nHasil Pencarian');
console.log(hasilCari);
```

```
Problems  Output  Debug Console  Terminal  Ports        powershell - hari-05-fungsi

PS E:\GOODIGITAL\portofolio-junior-web\hari-05-fungsi> node array-peserta.js
Hasil Pencarian
[
  { id: 1, nama: 'Amri', kelas: 'Web', nilai: 97 },
  { id: 4, nama: 'Tamam', kelas: 'Data', nilai: 87 },
  { id: 5, nama: 'Azzam', kelas: 'Web', nilai: 75 }
]
```

6. Daftar Peringkat

- Mengurutkan data peserta berdasarkan nilai.
- Membuat daftar peringkat menggunakan sort().
- Menampilkan nomor urut peserta.

```
const peringkat = [...peserta]
  .sort((a, b) => b.nilai - a.nilai);

console.log('\nDaftar Peringkat');

peringkat.forEach((p, index) => {
  console.log(
    `${index + 1}. ${p.nama} - ${p.nilai}`
  );
});
```

```
Problems  Output  Debug Console  Terminal  Ports        powershell - hari-05-fungsi

PS E:\GOODIGITAL\portofolio-junior-web\hari-05-fungsi> node array-peserta.js
Daftar Peringkat
1. Badru - 100
2. Haura - 99
3. Amri - 97
4. Abdul - 95
5. Tamam - 87
6. Aina - 86
7. Levan - 84
8. Azzam - 75
9. Quo - 64
10. Yaqin - 56
11. Dia - 40
```

7. Menghapus Duplikat Email dengan Set

- Membuat array email yang mengandung data duplikat.
- Menghapus data duplikat menggunakan Set.
- Menampilkan hasil data email unik.

```
const email = [
  'amri@gmail.com',
  'badru@gmail.com',
  'amri@gmail.com',
  'haura@gmail.com',
  'badru@gmail.com'
];

const emailUnik = [...new Set(email)];

console.log('\nEmail Unik');
console.log(emailUnik);
```

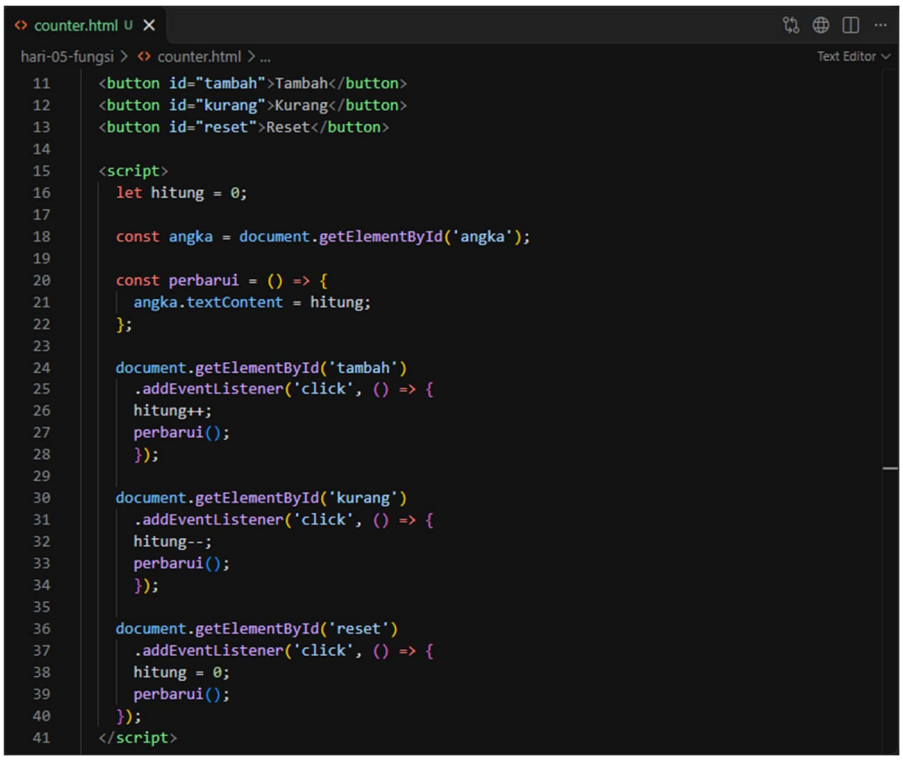
```
Problems  Output  Debug Console  Terminal  Ports        powershell - hari-05-fungsi

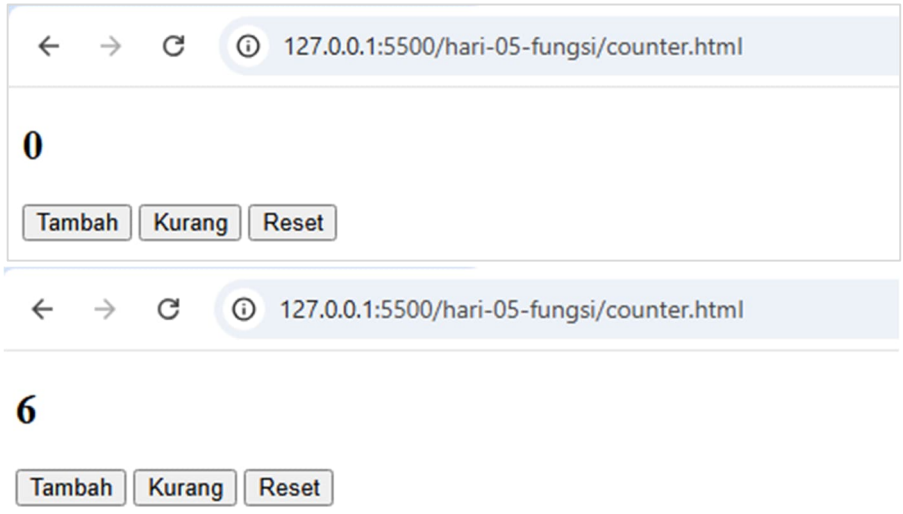
PS E:\GOODIGITAL\portofolio-junior-web\hari-05-fungsi> node array-peserta.js
[ 'amri@gmail.com', 'badru@gmail.com', 'haura@gmail.com' ]
```

Lab 3 – DOM dan Event

1. Counter Sederhana

- Membuat counter sederhana menggunakan DOM.
- Menambahkan event click pada tombol tambah, kurang, dan reset.
- Memperbarui nilai counter secara dinamis.

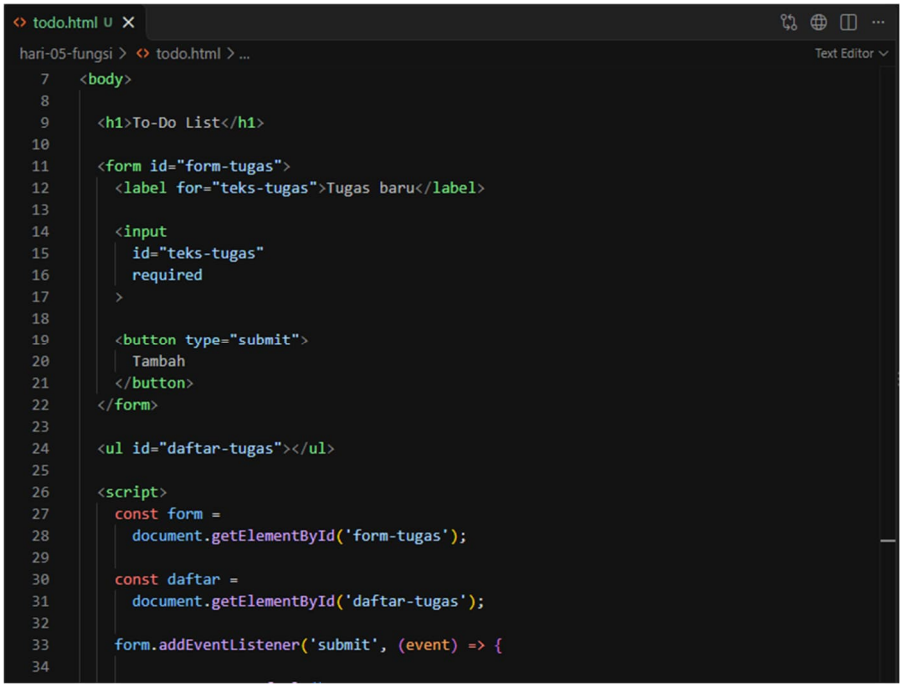


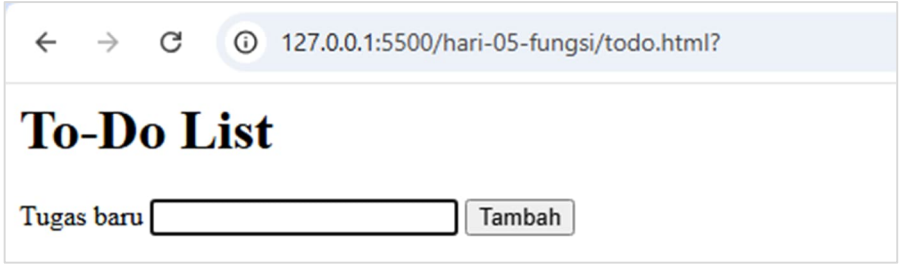


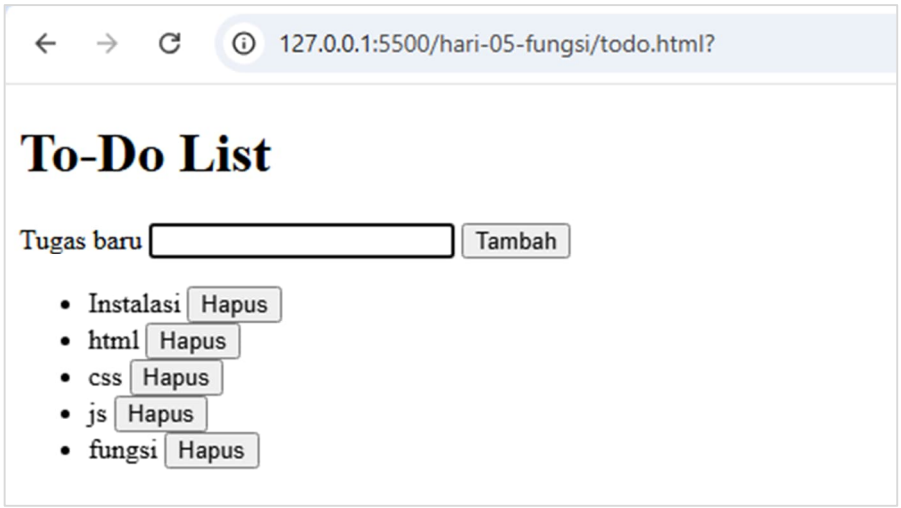
2. To-Do List Sederhana

- Membuat form input tugas.
- Menambahkan event click.

- Menambahkan item tugas ke halaman menggunakan DOM.







3. Tandai Tugas Selesai

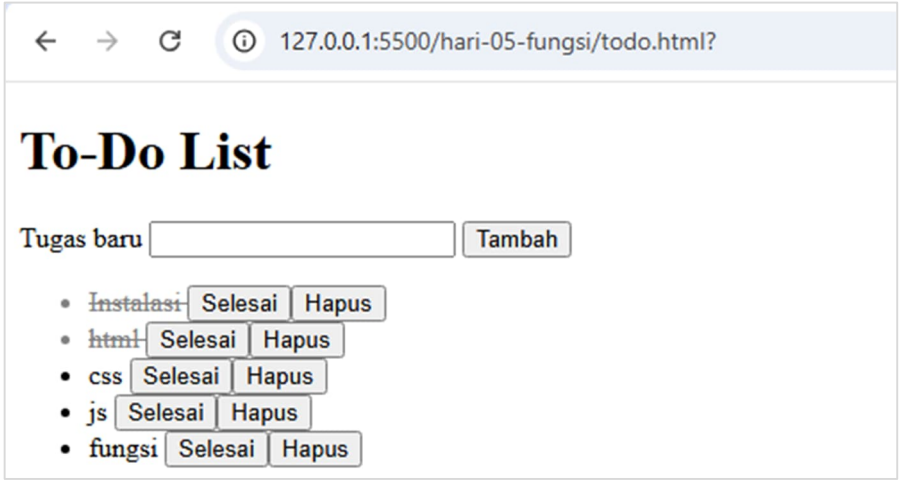
- Menambahkan status selesai pada tugas.
- Menggunakan classList.toggle().
- Mengubah tampilan tugas dengan CSS.

```html
<head>
  <style>
    .selesai {
      text-decoration: line-through;
      color: gray;
    }
  </style>
  <meta charset="UTF-8">
  <title>To-Do List</title>
</head>
```

```javascript
const selesai =
  document.createElement('button');

selesai.type = 'button';
selesai.textContent = 'Selesai';

selesai.addEventListener('click', () => {
  item.classList.toggle('selesai');
});
```



4. Jumlah Tugas Tersisa

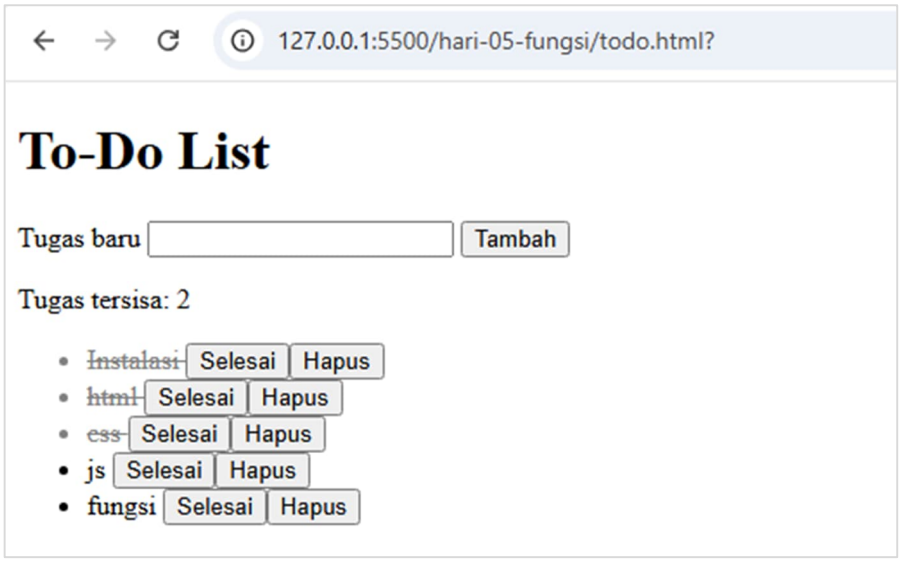
- Menghitung jumlah tugas aktif.
- Menampilkan jumlah tugas tersisa.
- Memperbarui jumlah secara otomatis.

```javascript
const jumlahTugas =
  document.getElementById('jumlah-tugas');

function perbaruiJumlahTugas() {

  const tersisa =
    daftar.querySelectorAll(
      'li:not(.selesai)'
    ).length;

  jumlahTugas.textContent = tersisa;
}
```

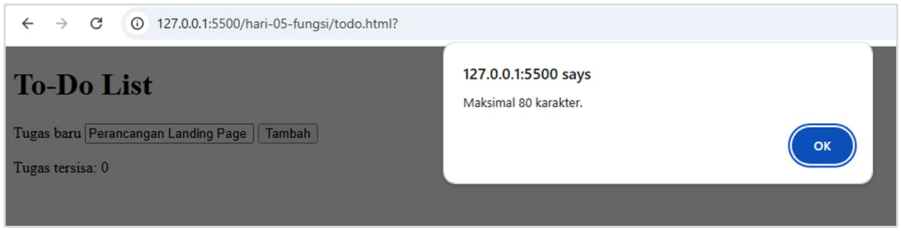


5. Batasi Maksimal 80 Karakter

- Membatasi panjang teks tugas.
- Menampilkan pesan validasi.
- Menguji batas karakter.

```
if (teks === '') {
  return;
}

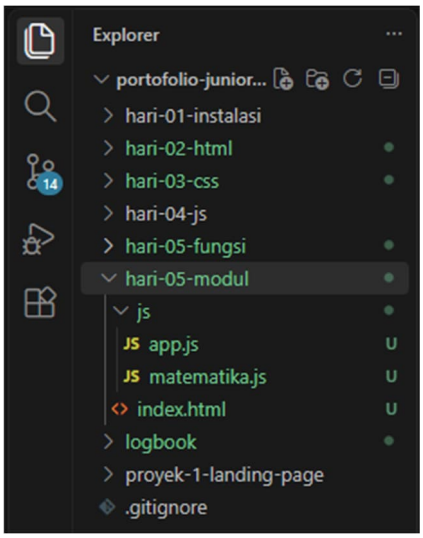
if (teks.length > 80) {
  alert('Maksimal 80 karakter.');
  return;
}
```



Lab 4 - Modul ES

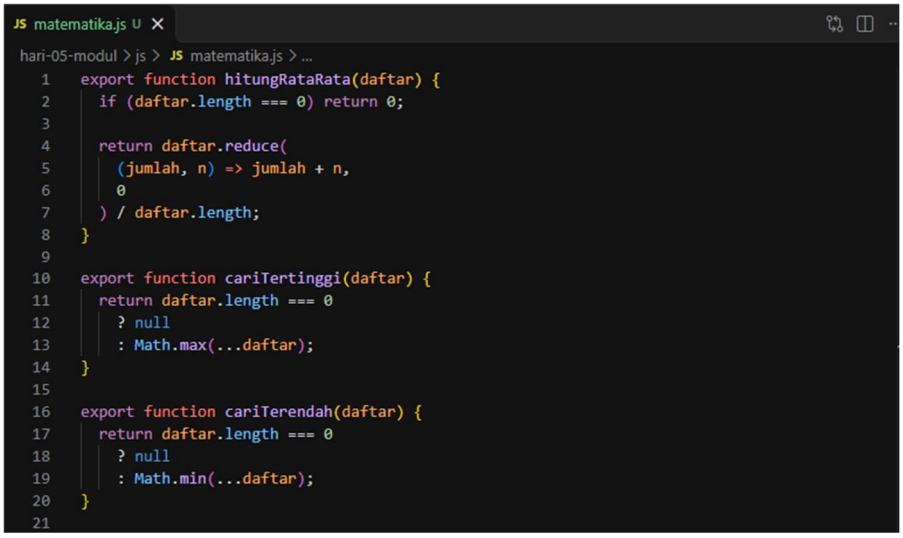
1. Struktur Proyek

- Membuat folder proyek modular.
- Membuat file HTML dan JavaScript.
- Menyiapkan struktur modul ES.



2. Modul Matematika

- Memindahkan fungsi statistik ke modul terpisah.
- Menggunakan export.



3. Antarmuka HTML

- Membuat input nilai.
- Menyiapkan tombol proses.
- Menyiapkan area hasil.

index.html U

hari-05-modul > index.html > ...

Text Editor

```html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Modul ES</title>
</head>
<body>

  <h1>Statistik Nilai</h1>

  <label for="nilai">
    Masukkan nilai (pisahkan dengan koma)
  </label>

  <input
    type="text"
    id="nilai"
    placeholder="80,90,75,100"
  >

  <button id="proses">
    Proses
  </button>

  <div id="hasil"></div>

  <script
    type="module"
    src="js/app.js">
  </script>
```

4. Import pada app.js

- Mengimpor fungsi dari modul.
- Mengolah data dari input pengguna.
- Menampilkan hasil pada halaman.

app.js U

hari-05-modul > js > app.js > ...

```js
import {
  hitungRataRata,
  cariTertinggi,
  cariTerendah
} from './matematika.js';

const tombol =
  document.getElementById('proses');

const hasil =
  document.getElementById('hasil');

tombol.addEventListener('click', () => {

  const input =
    document.getElementById('nilai').value;

  const daftar =
    input
      .split(',')
      .map((n) => Number(n.trim()))
      .filter((n) => !Number.isNaN(n));

  hasil.innerHTML = `
    <p>Rata-rata : ${hitungRataRata(daftar).toFixed(2)}</p>
    <p>Tertinggi : ${cariTertinggi(daftar)}</p>
    <p>Terendah : ${cariTerendah(daftar)}</p>
  `;
});
```

5. Live Server

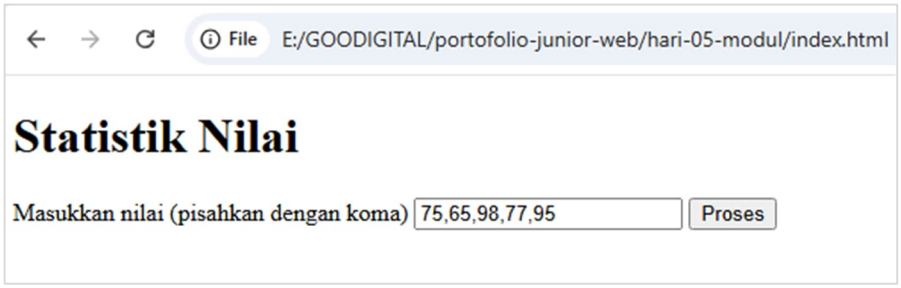
- Menjalankan proyek menggunakan Live Server.

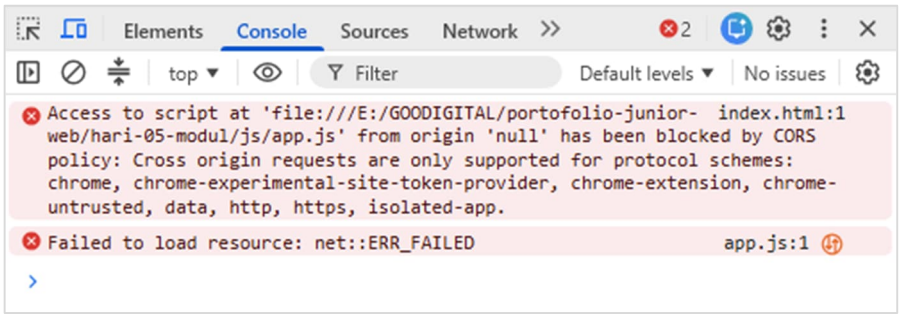
- Menguji proses import/export modul.
- Mengamati hasil pada browser.



6. Analisis Error file:///

- Menjalankan aplikasi menggunakan file:///.
- Mengamati pesan error pada browser.
- Menjalankan aplikasi menggunakan Live Server.
- Menganalisis penyebab perbedaan hasil eksekusi.
- Modul ES (import dan export) memerlukan web server untuk memuat file JavaScript antar modul. Saat menggunakan protokol file:///, browser membatasi akses tersebut karena kebijakan keamanan (CORS). Oleh karena itu aplikasi modular harus dijalankan melalui server seperti Live Server atau XAMPP agar modul dapat dimuat dengan benar.





Bug Hunt

1. Analisis Bug

- Mengidentifikasi kesalahan pada fungsi dan DOM.
- Menganalisis penyebab bug.
- Memperbaiki kode yang bermasalah.
- Menguji hasil perbaikan.



```
function tambah(a, b) {
  a + b;
}

console.log(tambah(5, 3));

function hitungRataRata(daftar) {
  return daftar.reduce((j, n) => j + n, 0) / daftar.length;
}

console.log(hitungRataRata([]));

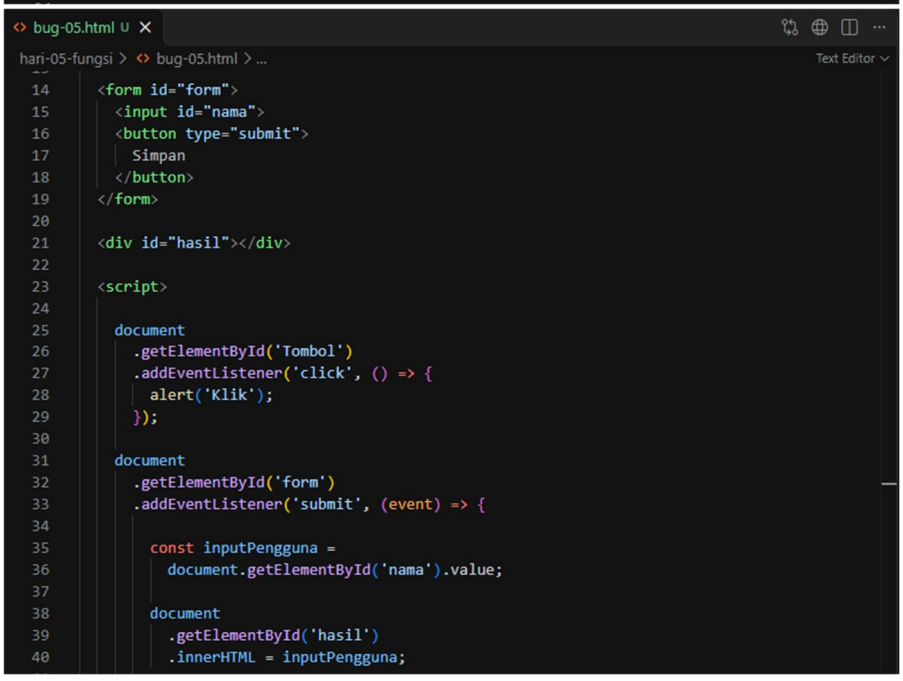
const angka = [10, 2, 5];

angka.sort();

console.log(angka);

const tombol = [];

for (var i = 0; i < 3; i++) {
  tombol.push(() => console.log(i));
}
```



```
<form id="form">
  <input id="nama">
  <button type="submit">
    Simpan
  </button>
</form>

<div id="hasil"></div>

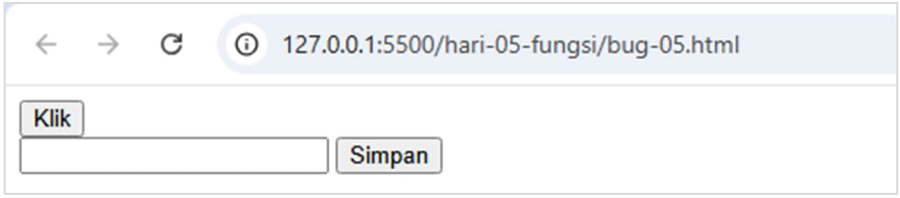
<script>

  document
    .getElementById('Tombol')
    .addEventListener('click', () => {
      alert('Klik');
    });

  document
    .getElementById('form')
    .addEventListener('submit', (event) => {

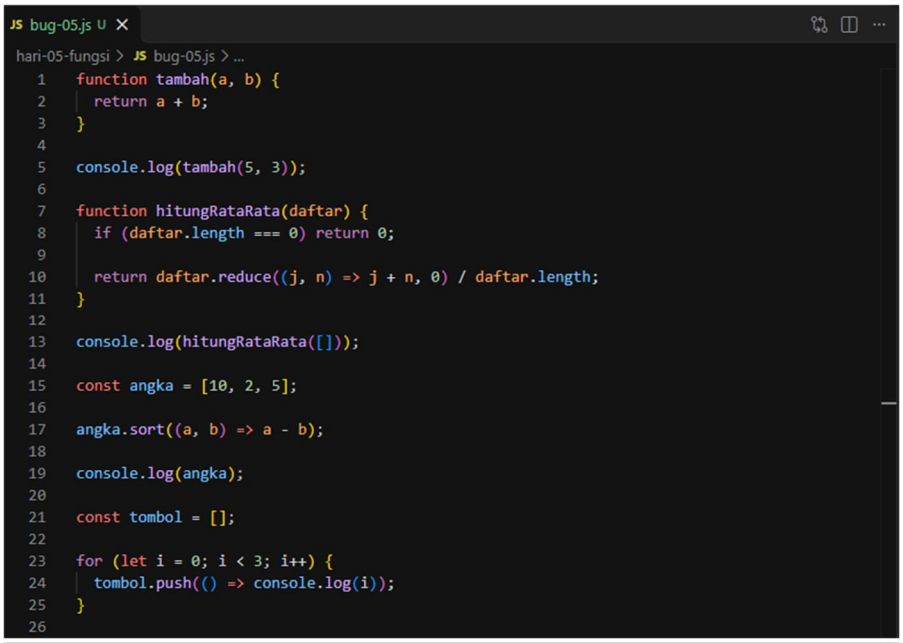
      const inputPengguna =
        document.getElementById('nama').value;

      document
        .getElementById('hasil')
        .innerHTML = inputPengguna;
```



2. Perbaikan Bug

- Menjalankan file hasil perbaikan.
- Memverifikasi fungsi dan DOM berjalan normal.
- Memastikan seluruh bug telah diperbaiki.



```
function tambah(a, b) {
  return a + b;
}

console.log(tambah(5, 3));

function hitungRataRata(daftar) {
  if (daftar.length === 0) return 0;

  return daftar.reduce((j, n) => j + n, 0) / daftar.length;
}

console.log(hitungRataRata([]));

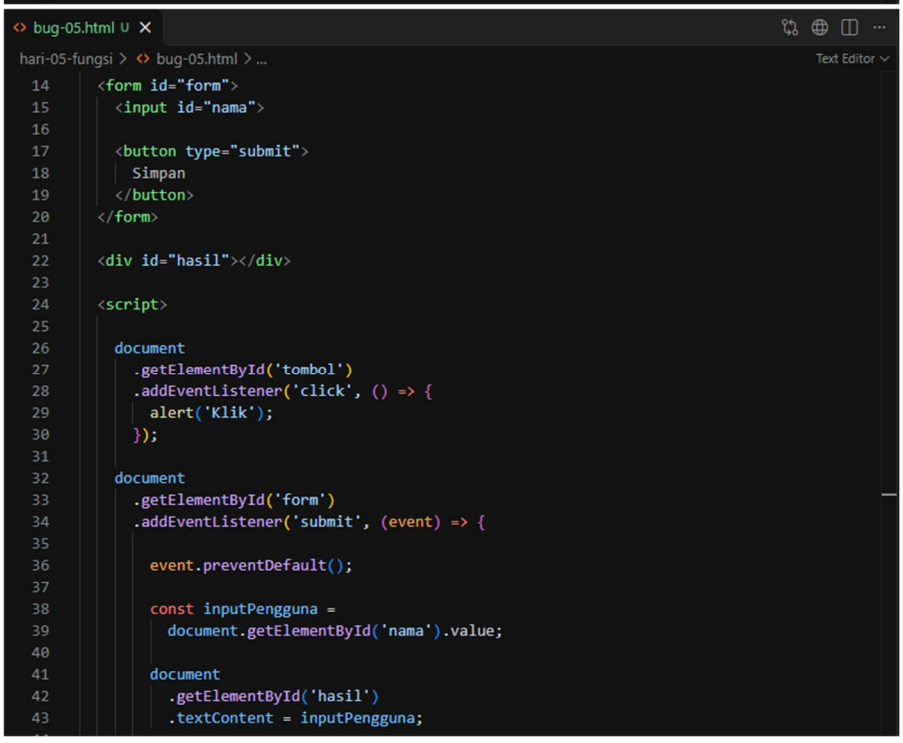
const angka = [10, 2, 5];

angka.sort((a, b) => a - b);

console.log(angka);

const tombol = [];

for (let i = 0; i < 3; i++) {
  tombol.push(() => console.log(i));
}
```



```
<form id="form">
  <input id="nama">

  <button type="submit">
    Simpan
  </button>
</form>

<div id="hasil"></div>

<script>

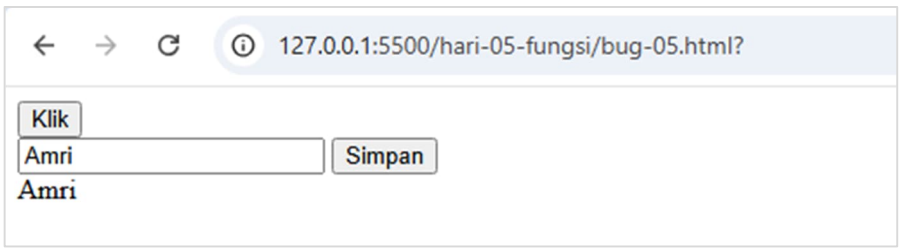
  document
    .getElementById('tombol')
    .addEventListener('click', () => {
      alert('Klik');
    });

  document
    .getElementById('form')
    .addEventListener('submit', (event) => {

      event.preventDefault();

      const inputPengguna =
        document.getElementById('nama').value;

      document
        .getElementById('hasil')
        .textContent = inputPengguna;
```



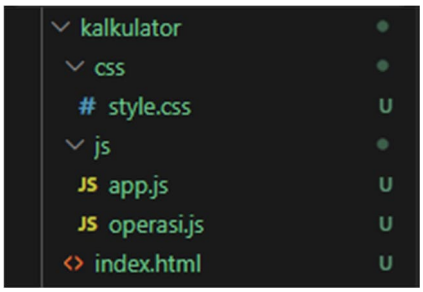
No	Gejala	Penyebab	Perbaikan
1	Hasil fungsi bernilai undefined	Fungsi tidak mengembalikan nilai	Tambahkan return
2	Hasil rata-rata menjadi NaN	Array kosong tidak ditangani	Tambahkan pengecekan array kosong
3	Muncul TypeError saat menambahkan event	ID elemen tidak sesuai huruf besar-kecil	Samakan penulisan id
4	Elemen tidak ditemukan saat script dijalankan	Script dimuat sebelum elemen HTML tersedia	Pindahkan script ke akhir body atau gunakan defer
5	Halaman refresh setelah submit	Tidak menggunakan event.preventDefault()	Tambahkan event.preventDefault()
6	Urutan angka tidak sesuai	sort() membandingkan sebagai string	Gunakan sort((a, b) => a - b)
7	Semua tombol menampilkan nilai yang sama	Menggunakan var di dalam loop	Ganti var menjadi let
8	Input pengguna dapat disisipkan HTML	Menggunakan innerHTML	Gunakan textContent

PBL Hari 5 - Proyek 2: Kalkulator dan Form Validator

Bagian A - Kalkulator Modular

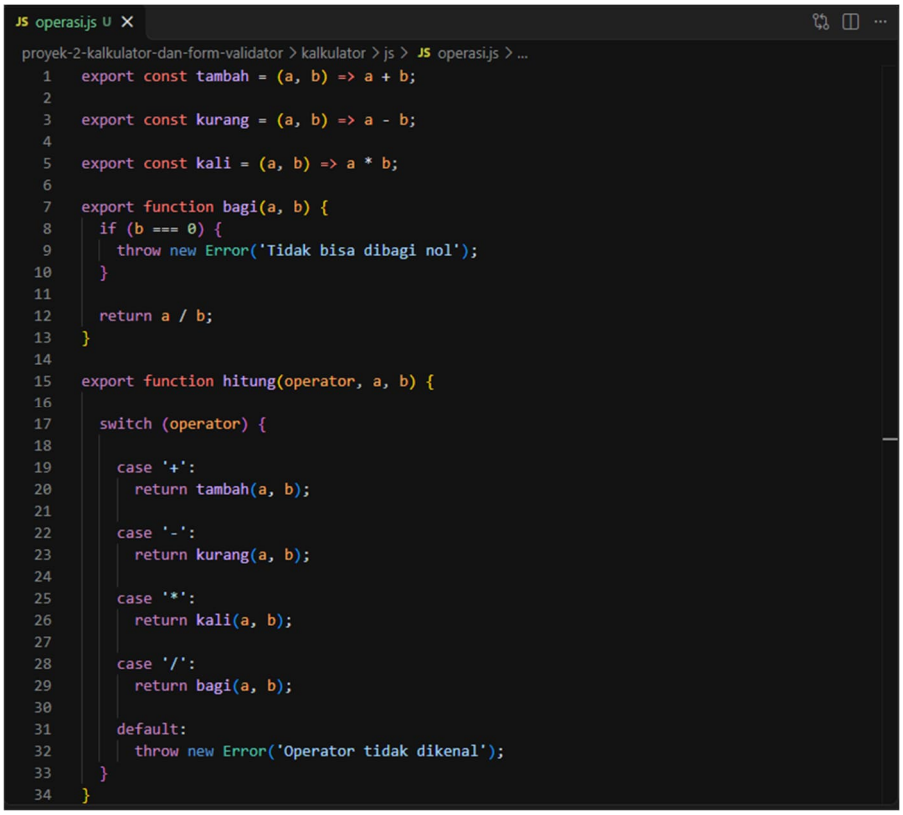
1. Struktur Proyek

- Membuat struktur folder kalkulator.
- Membuat file HTML, CSS, dan JavaScript.
- Menyiapkan modul operasi.



2. Modul Operasi

- Membuat fungsi operasi hitung.
- Membuat fungsi pembagian dengan validasi.
- Membuat fungsi hitung().



3. Antarmuka Kalkulator

- Membuat layar kalkulator.
- Membuat tombol angka dan operator.
- Menyusun tombol menggunakan CSS Grid.

index.html U

proyek-2-kalkulator > index.html > ...

Text Editor

```
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <main class="kalkulator">

    <div
      id="layar"
      class="layar"
      aria-live="polite">
      0
    </div>

    <div id="papan" class="papan">

      <button data-aksi="hapus">C</button>

      <button data-operator="/">/</button>

      <button data-operator="*">x</button>
```

style.css U

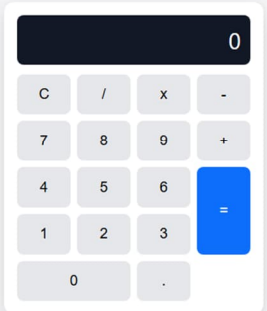
proyek-2-kalkulator > css > style.css > ...

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  display: grid;
  place-items: center;
  min-height: 100vh;
  margin: 0;
  background: #f3f4f6;
}

.kalkulator {
  width: min(320px, 92vw);
  padding: 1rem;
  background: #fff;
  border-radius: 12px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
}
```

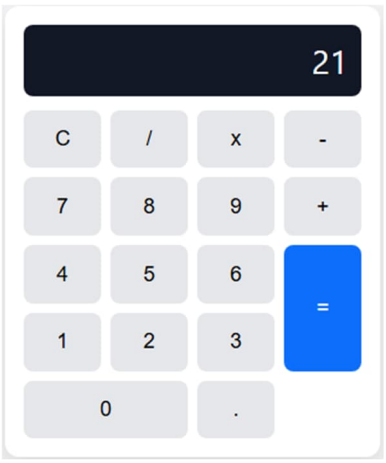
127.0.0.1:5500/proyek-2-kalkulator/index.html



4. Menghubungkan UI dan Logika

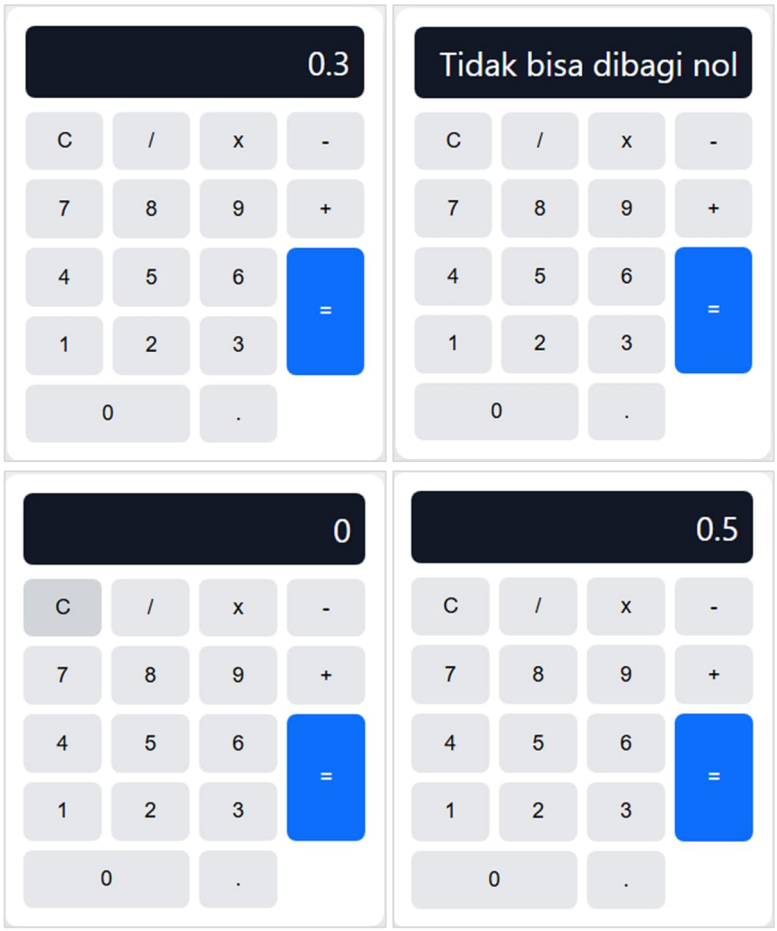
- Mengimpor fungsi dari operasi.js.
- Menangani input angka dan operator.
- Menampilkan hasil perhitungan pada layar.

```
JS app.js U ×
proyek-2-kalkulator > js > JS app.js > ...
1  import { hitung } from './operasi.js';
2
3  const layarEl = document.getElementById('layar');
4  const papan = document.getElementById('papan');
5
6  let layar = '0';
7  let angkaPertama = null;
8  let operator = null;
9  let siapReset = false;
10
11 function perbaruiLayar() {
12   layarEl.textContent = layar;
13 }
14
15 function masukkanAngka(digit) {
16   if (digit === '.' && layar.includes('.') && !siapReset) return;
17
18   if (layar === '0' && digit !== '.') {
19     layar = digit;
20   } else if (siapReset) {
21     layar = digit === '.' ? '0.' : digit;
22     siapReset = false;
23   } else {
24     layar += digit;
25   }
26 }
27
28 function hitungHasil() {
29   if (operator === null) return;
30
```



5. Pengujian Kasus Tepi

- Menguji angka desimal.
- Menguji pembagian dengan nol.
- Menguji penekanan operator berulang.
- Menguji input angka yang diawali titik.



6. Hapus Satu Karakter

- Menambahkan tombol hapus karakter.
- Menghapus digit terakhir pada layar.
- Menguji fungsi hapus karakter.

```
<div id="papan" class="papan">

  <button data-aksi="hapus">C</button>
  <button data-operator="/">/</button>
  <button data-operator="*">x</button>
  <button data-aksi="hapus-satu">⌫</button>
```

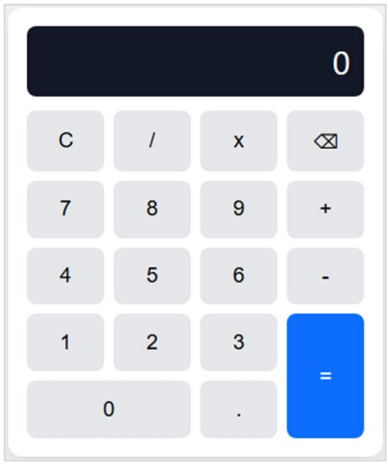
```
function hapusSatuKarakter() {

  if (layar.length <= 1) {
    layar = '0';
    return;
  }

  layar = layar.slice(0, -1);
}
```

```
else if (
  tombol.dataset.aksi === 'hapus'
) {
  bersihkan();
}

else if (
  tombol.dataset.aksi === 'hapus-satu'
) {
  hapusSatuKarakter();
}
```



7. Riwayat 5 Perhitungan Terakhir

- Menyimpan hasil perhitungan ke array.
- Menampilkan 5 riwayat terakhir.
- Menguji penyimpanan riwayat.

```
<button
  id="tampil-riwayat"
  class="riwayat-btn">
  Riwayat
</button>

  <ul id="riwayat" hidden></ul>
```

```
.riwayat-btn {
  width: 100%;
  margin-bottom: 0.75rem;
  padding: 0.5rem;
  border: 0;
  border-radius: 8px;
  background: #6c757d;
  color: white;
  cursor: pointer;
  font-size: 0.9rem;
}

.riwayat-btn:hover {
  background: #0b5ed7;
}
```

```
const riwayat = [];
const tombolRiwayat =
  document.getElementById('tampil-riwayat');

const daftarRiwayat =
  document.getElementById('riwayat');

  tombolRiwayat.addEventListener('click', () => {
  daftarRiwayat.hidden =
    !daftarRiwayat.hidden;
  });
```

```
function tampilkanRiwayat() {

  daftarRiwayat.innerHTML = '';

  riwayat.forEach((item) => {

    const li =
      document.createElement('li');

    li.textContent = item;

    daftarRiwayat.appendChild(li);
  });
}
```



8. Dukungan Keyboard

- Menambahkan event keydown.
- Menghubungkan keyboard dengan tombol kalkulator.
- Menguji input menggunakan keyboard.

```
document.addEventListener('keydown', (event) => {

  const tombol = event.key;

  if ('0123456789.'.includes(tombol)) {
    masukkanAngka(tombol);
  }

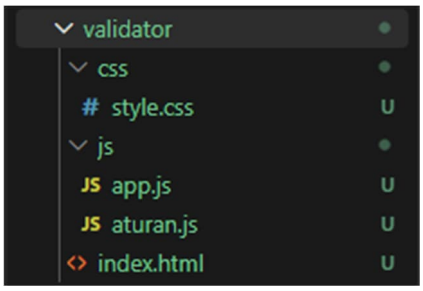
  else if ('+-*/'.includes(tombol)) {
    pilihOperator(tombol);
  }

  else if (tombol === 'Enter') {
    event.preventDefault();
    hitungHasil();
  }
```

Bagian B - Form Validator

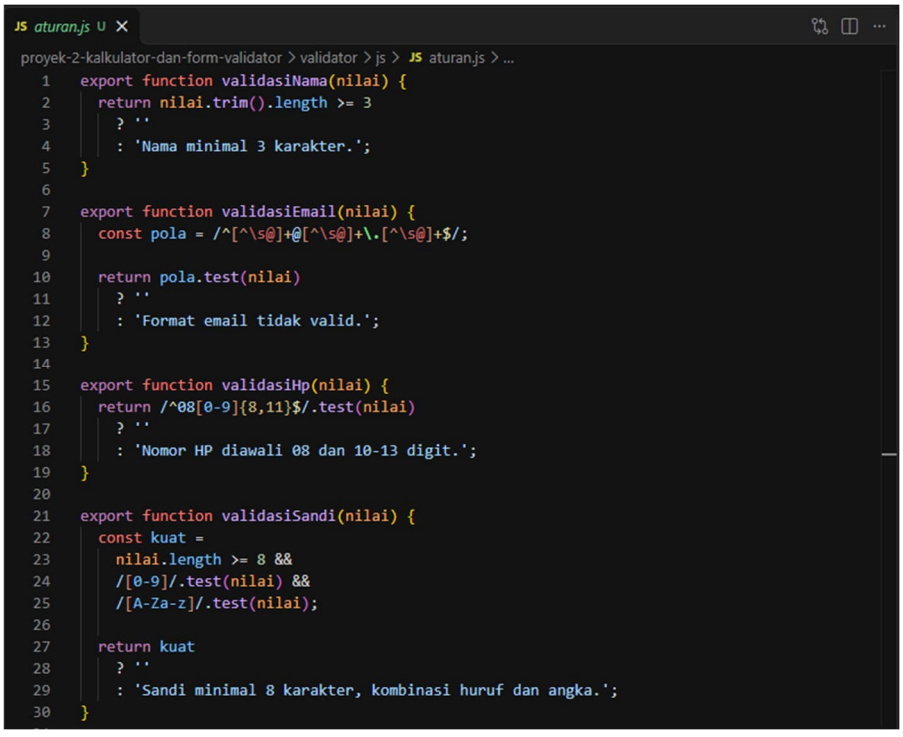
1. Struktur Proyek

- Membuat struktur folder validator.
- Membuat file HTML, CSS, dan JavaScript.
- Menyiapkan modul validasi.



2. Modul Validasi

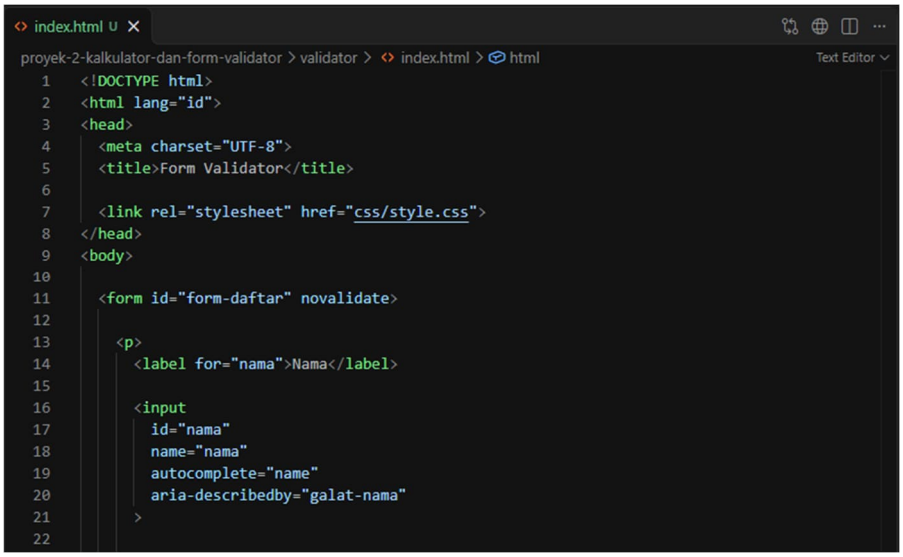
- Membuat fungsi validasi nama.
- Membuat fungsi validasi email.
- Membuat fungsi validasi nomor HP.
- Membuat fungsi validasi sandi dan konfirmasi.

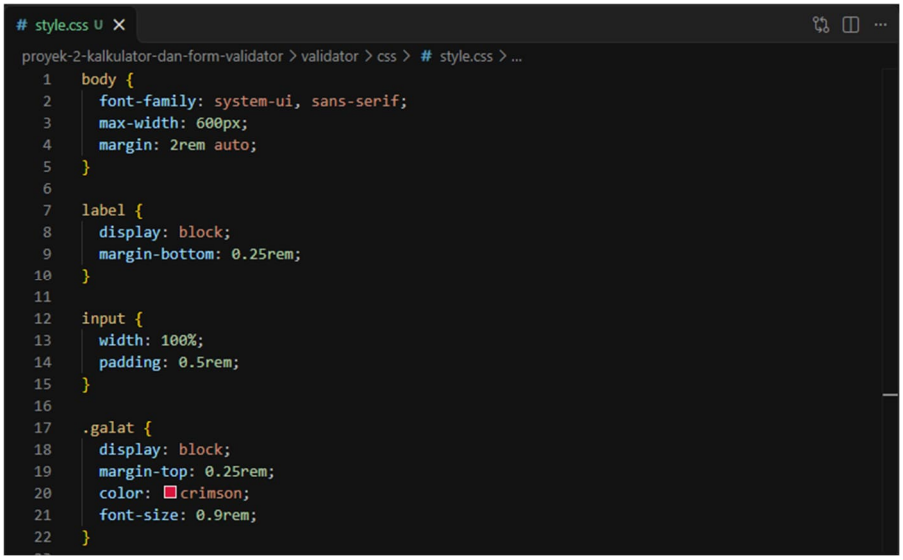


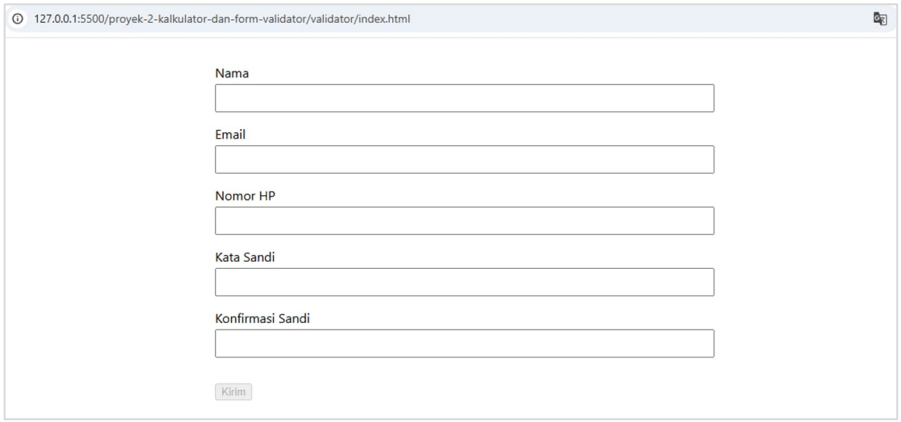
3. Form Pendaftaran

- Membuat form pendaftaran akun.
- Menambahkan field nama, email, nomor HP, sandi, dan konfirmasi sandi.

- Menyiapkan area pesan validasi.
- Menyiapkan tombol kirim.



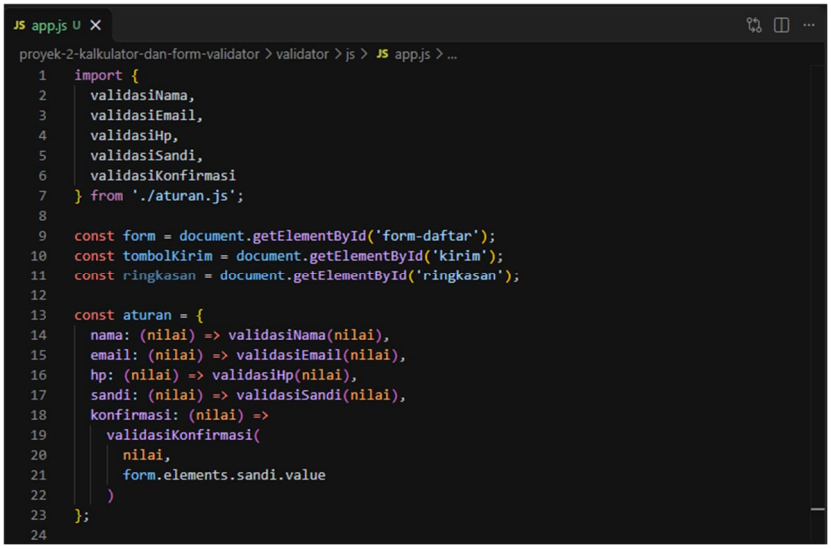




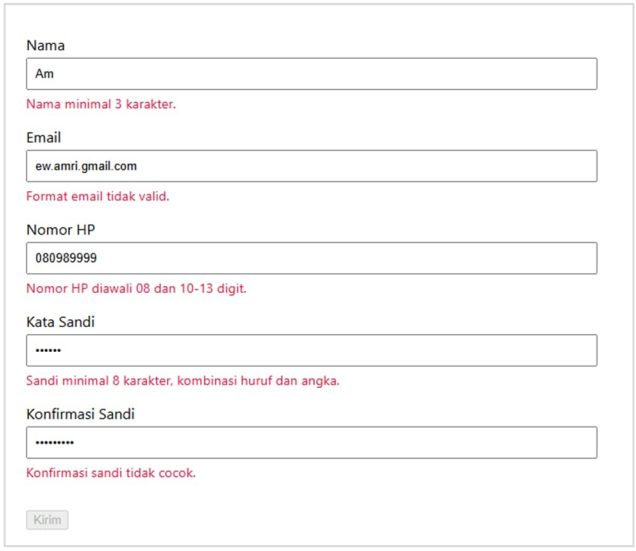
4. Validasi Real-Time

- Mengimpor fungsi validasi dari aturan.js.
- Menampilkan pesan validasi saat pengguna mengetik.

- Mengaktifkan tombol kirim hanya jika seluruh data valid.



```
app.js U
proyek-2-kalkulator-dan-form-validator > validator > js > app.js > ...
1  import {
2    validasiNama,
3    validasiEmail,
4    validasiHp,
5    validasiSandi,
6    validasiKonfirmasi
7  } from './aturan.js';
8
9  const form = document.getElementById('form-daftar');
10 const tombolKirim = document.getElementById('kirim');
11 const ringkasan = document.getElementById('ringkasan');
12
13 const aturan = {
14   nama: (nilai) => validasiNama(nilai),
15   email: (nilai) => validasiEmail(nilai),
16   hp: (nilai) => validasiHp(nilai),
17   sandi: (nilai) => validasiSandi(nilai),
18   konfirmasi: (nilai) =>
19     validasiKonfirmasi(
20       nilai,
21       form.elements.sandi.value
22     )
23 };
24
```





5. Ringkasan Data Setelah Submit

- Memvalidasi seluruh field saat form dikirim.
- Menampilkan ringkasan data pendaftaran.
- Mereset form setelah berhasil dikirim.

```
JS app.js U ×
proyek-2-kalkulator-dan-form-validator > validator > js > JS app.js > ...
76
77  form.addEventListener('submit', (event) => {
78
79    event.preventDefault();
80
81    const hasil =
82      Object.keys(aturan).map(periksaField);
83
84    if (!hasil.every(Boolean)) {
85      return;
86    }
87
88    ringkasan.hidden = false;
89
90    ringkasan.textContent =
91      `Pendaftaran berhasil: ${form.elements.nama.value} (${form.elements.email.value})`;
92
93    form.reset();
94
95    tombolKirim.disabled = true;
96
97  });
98
```



6. Review

Butir	Ya/Tidak	Catatan
Tidak ada fungsi lebih dari sekitar 20 baris	☑ Ya	Fungsi sudah dipisah-pisah sehingga relatif pendek dan mudah dibaca.
Tidak ada duplikasi kode (DRY)	☑ Ya	Sudah memanfaatkan fungsi seperti tampilkanMenu(), tampilkanLaporan(), dan modul terpisah.
Event dipasang dengan addEventListener, bukan atribut inline	☑ Ya	Event sudah menggunakan addEventListener() pada seluruh fitur yang diuji.
Data pengguna ditampilkan dengan textContent	☑ Ya	Pada form validator sudah menggunakan textContent sehingga lebih aman.
Kasus tepi diuji dan terdokumentasi	☑ Ya	Sudah menguji pembagian dengan nol, angka desimal, operator berulang, dan validasi form.
Fungsi publik memiliki JSDoc	☑ Ya	Fungsi utama sudah diberi dokumentasi JSDoc.

7. Repository GitHub

- Melakukan commit.
- Melakukan push ke GitHub.
- Tautan repository: https://github.com/amribaros15/portofolio-junior-web/

